1 Reference plans and the cost model

1.1 Proven-optimal reference plans

To say how far an LLM's plan is from the best possible plan we need the optimum of every instance, not just a good plan. A reference that is itself sub-optimal makes the comparison misleading: against our first references, the direct plans of each Claude model appeared to *beat* the reference on 20 of the 22 field reports for the 11 instances that were not yet proven optimal, which says more about the reference than about the LLM.

Fast Downward alone is not enough. We first solved every instance with Fast Downward [1], using the optimal configurations A* with LM-cut [2] and A* with merge-and-shrink (M&S) [3], 300 s each, falling back to anytime LAMA [5] when both ran out of time. Table 1 shows that this proves every small instance and 14 of 15 medium ones optimal, but only 5 of 15 large ones: 34 of 45 overall. For the 11 unproven instances LAMA's best plan was 17–88% above the optimum (Table 2). Taking the cheaper of the splitting and no-split domains, which was our reference before the exact solver, narrowed this to 0–45%, but those references still could not be shown to be optimal.

The difficulty comes from the encoding rather than from the size of the road network. Counts of people are level objects with a static `(sum a b c)` relation, so `load` has ten parameters and grounds to many actions, and the same set of rescues can be reached through many orders of loads, drives and car assignments. The heuristics are also weak on this domain: under the delete relaxation a car keeps every `free-seats` value it has ever had, so seat capacity disappears and one relaxed tour can pick up everybody. LM-cut therefore ignores the repeated trips that make up most of the real cost on large instances. Merge-and-shrink expanded up to 29.5 million states (medium-04) before proving optimality, and LAMA expanded up to 3.3 million (large-15) without reaching the optimum.

An exact domain-specific solver. We therefore wrote `exact.py`, a uniform-cost (Dijkstra) search over a much smaller state space. Write $G = (V, E)$ for the graph of undamaged roads with travel times $t(e)$, $d(u, v)$ for shortest-path distances (Floyd–Warshall), $Q = 4$ for seats per car and p_b for the deferral penalty of band b. The objective is the total driving time plus $\sum p_b$ over everyone deferred. Five properties of the domain let us shrink the search without losing optimality:

1. *Shortest paths.* Between two stops (a pick-up or a drop-off) a car should follow a shortest path, so driving is precomputed as $d(u, v)$.

2. *Trips.* Unloading is free, only possible at evacuation points, and every car must end empty, so each car's route is a sequence of trips: from its current position, visit a set S of pick-up sites, then drive to an evacuation point e. A trip costs the shortest route through S to e, found by trying every order ($|S| \le Q$, so at most 24 orders).

3. *Cars one at a time.* The model has no clock, the cost is a plain sum, and cars only interact through who is still waiting. Any joint plan can be reordered so car 1 does all its trips, then car 2, at the same cost.

4. *Most urgent first.* At one site, taking a higher-penalty person instead of a lower-penalty one never raises the cost, so we only need to know *how many* people n_v have been taken from site v; they are taken in decreasing order of p_b.

	Fast Downward		`exact.py`		
Size	proven	by LM-cut / M&S	proven	max time (s)	max states
Small	15/15	15 / 0	15/15	<0.1	47
Medium	14/15	10 / 4	15/15	0.4	5,197
Large	5/15	2 / 3	15/15	109.0	242,399
All	34/45		45/45		

Table 1: Proving the 45 reference plans optimal. Fast Downward ran A* with LM-cut, then A* with merge-and-shrink, with a 300 s limit for each search. `exact.py` times are single-threaded Python on one core.

Instance	Optimum	FD split (LAMA)	gap	Old reference	gap
medium-14	401	471	17%	401	0%
large-01	240	452	88%	268	12%
large-02	428	721	68%	461	8%
large-03	434	602	39%	542	25%
large-06	445	646	45%	446	0%
large-08	366	575	57%	430	17%
large-10	402	616	53%	480	19%
large-11	403	586	45%	529	31%
large-12	302	437	45%	437	45%
large-13	212	389	83%	250	18%
large-15	476	695	46%	487	2%
Mean			53%		16%

Table 2: The 11 instances Fast Downward could not prove optimal in 300 s, with the optimum found by `exact.py`. "Old reference" is the cheaper of the Fast Downward plans for the splitting and no-split domains, which we used before the exact solver.

5. *Full cars.* Taking one more person on a trip that already visits their site adds no driving and removes a penalty, so a trip through S carries $\min(Q, \sum_{v \in S} \text{waiting}_v)$ people.

A state is then $(c, pos, (n_v)_v)$: the car being planned, where it is, and how many people have been taken from each site. The successors are every trip of car c (cost: its driving), handing over to car $c{+}1$ (cost 0), and, after the last car, deferring everyone left (cost: their penalties). All edge costs are non-negative, so Dijkstra returns an optimal plan. We do not need a heuristic: the reduced space has at most 242,399 states (large-03), against the millions Fast Downward expanded.

Checking the solver. Because `exact.py` is our own code, we checked it in two independent ways. It returns exactly the same cost as Fast Downward on all 34 instances that Fast Downward proved optimal, and every plan it outputs is accepted by our plan checker (which agrees with VAL) with the claimed cost. All 45 instances solve in about four minutes in total, most in under a second (Table 1); the per-instance optima are listed in Appendix A. These 45 proven optima are the references used throughout our evaluation.

What this says about the tool. The exact solver is not a replacement for a planner in our comparison: it is hand-written for this one domain, and the reductions above stop being exact as soon as the model gains a clock (deadlines, or cars that must coordinate in time), non-identical

seats, or a cost to unload. It does show that the hard part of these instances is search, not modelling: a general planner given a correct PDDL model still fails to prove optimality on 11 of 45 instances within five minutes, while a solver that exploits the structure of the problem needs seconds. This is the cost of generality the PDDL pipeline pays, and it matters when we compare it with the LLM, which plans directly and can use the same kind of structure implicitly.

1.2 Choosing the deferral penalty

Every person must end the plan either delivered to an evacuation point or deferred to a later rescue wave. Following the compilation of soft goals into action costs of Keyder and Geffner [4], deferring a person of band b costs a penalty p_b in the same unit as driving (minutes), and the planner minimises

$$\text{cost} \;=\; \sum_{\text{drives}} t(e) \;+\; \sum_{\text{deferred people}} p_b .$$

We write the penalty as a product of two parts that can each be justified on their own:

$$p_b = \lambda \cdot r_b, \qquad r_{\text{injured}} = 4,\; r_{\text{child}} = 3,\; r_{\text{elderly}} = 2.5,\; r_{\text{adult}} = 1.$$

The urgency weights r_b fix the *order* of priority, taken from triage practice (injured first, then children, then the elderly, then adults), relative to an adult. The scale λ is an exchange rate: the number of driving minutes the team is willing to spend to rescue one adult now rather than leave them for the next wave. Roughly, an optimal plan rescues a group when the extra driving needed to fetch them is less than the penalties λr_b it would otherwise pay. We use $\lambda = 20$, which gives penalties of 80, 60, 50 and 20 minutes.

Why λ matters. λ decides what kind of problem the benchmark is. At $\lambda = 0$ deferring is free and the optimal plan defers everyone without moving. When λ is large, deferring is never worth it, every reachable person is rescued, and the problem becomes pure capacitated vehicle routing with no triage left in it. (Minimising the number of people deferred first and only then the driving, a lexicographic objective we also considered, behaves like this large-λ limit.) The interesting regime, where the planner has to decide *who* to leave behind, lies in between. To find it we re-solved all 45 instances to optimality with `exact.py` (Section 1.1) for $\lambda \in \{0, 5, 10, 15, 20, 30, 40, 60, 80, 120\}$, keeping r_b fixed. People whom no car can reach because of damaged roads are deferred at every λ, so we leave them out of the counts below.

What the sweep shows. Figure 1 has the same shape for all three sizes: a convex curve where each extra person rescued costs more driving than the last. Going from $\lambda = 10$ to 20 rescues 160 more people for about 3,550 extra minutes of driving (about 22 minutes each); going from 20 to 40 rescues 112 more for about 3,940 minutes (about 35 minutes each). From $\lambda = 60$ upwards almost everyone reachable is rescued (one child in one small instance remains at $\lambda = 60$, none at $\lambda = 80$) and the curve stops moving.

At $\lambda = 20$ the plans rescue 512 of the 646 reachable people (79%) using 60% of the driving needed to rescue all of them, so the instances still require real triage decisions. Figure 2 shows that those decisions follow the intended priorities: of the 134 people left behind, 113 (84%) are adults, 13 elderly, 7 children and 1 injured person (on a large instance, where reaching them costs more than the 80-minute penalty). As λ grows the bars shrink in priority order: no reachable injured person is left from $\lambda = 30$, no elderly person from $\lambda = 40$ and no adult from $\lambda = 60$. The one

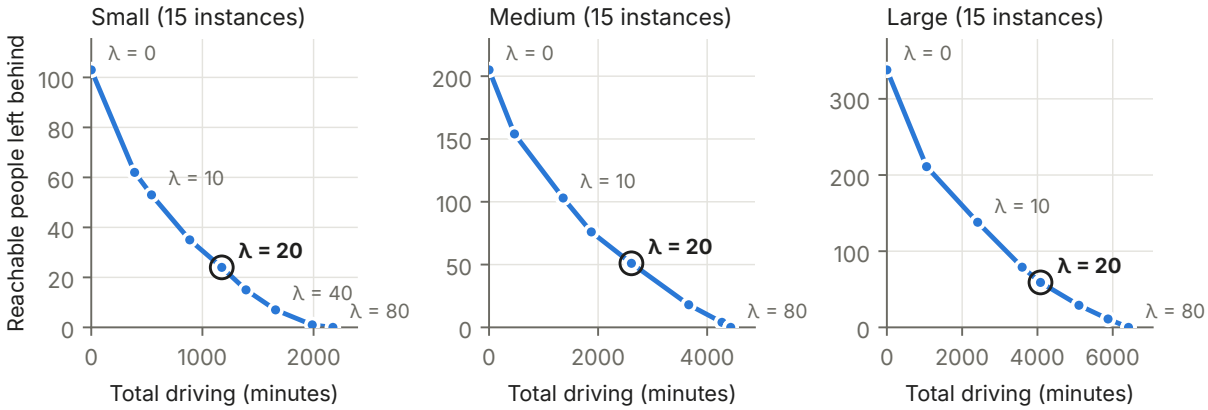


Figure 1: Trade-off between total driving and reachable people left behind, one point per value of λ (sum over the 15 instances of each size; every point is a proven optimum). $\lambda = 20$, used in all our experiments, is circled.

exception is a single child in one small instance whose rescue needs between 180 and 240 minutes of extra driving, so they are rescued only from $\lambda = 80$.

We chose $\lambda = 20$ for three reasons. It lies in the middle of the trade-off curve on every size, so instances are neither trivial (defer everyone, or rescue everyone) nor dominated by one regime. It is the regime that tests what we want to compare: whether an LLM, planning directly or through PDDL, makes the same who-to-leave decisions as the optimum. And at this value the plans the planner prefers are ones a human coordinator would recognise, since the people left behind are overwhelmingly the least urgent. All main results use $\lambda = 20$; other values appear only in this analysis.

Limitations and responsible use. λ and r_b encode a value judgement, how much driving a rescue is worth and how much more a child is worth than an adult, and our numbers are modelling choices rather than values from disaster-management practice. The formulation makes that judgement explicit and adjustable, which is a strength over leaving it implicit in an LLM's reasoning: an operator can see exactly what λ trades away, and Figure 1 shows the consequences of other choices. In a real deployment λ and r_b should be set and signed off by the responsible humans, not tuned by the system, and a deferred person should be read as "rescued by the next wave", not abandoned. Our model also has no clock, so it cannot express that waiting longer is worse for the injured than for others; a time-dependent penalty would capture this but makes the problem much harder to solve exactly.

References

[1] Malte Helmert. The Fast Downward planning system. *Journal of Artificial Intelligence Research*, 26:191–246, 2006.

[2] Malte Helmert and Carmel Domshlak. Landmarks, critical paths and abstractions: What's the difference anyway? In *Proceedings of the 19th International Conference on Automated Planning and Scheduling (ICAPS)*, pages 162–169, 2009.

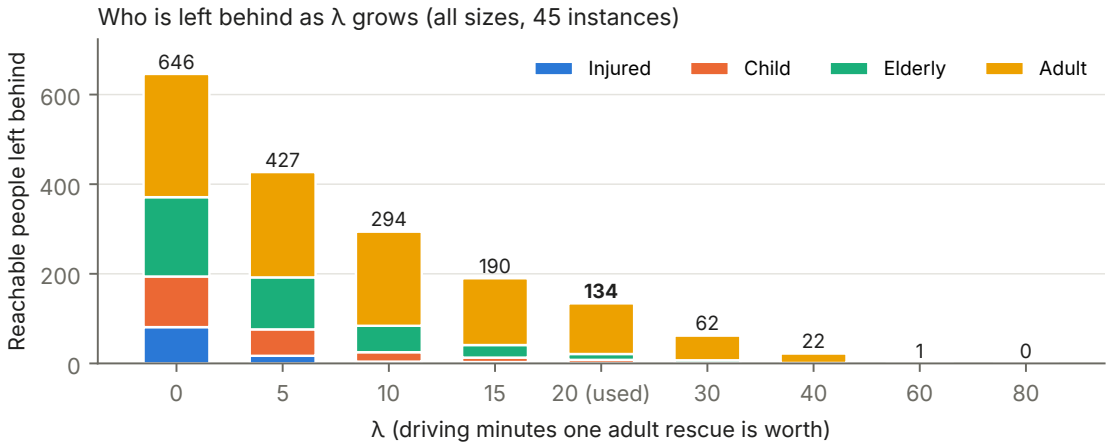


Figure 2: Reachable people left behind by band as λ grows, over all 45 instances. The urgency weights r_b are fixed; only the scale λ changes.

[3] Malte Helmert, Patrik Haslum, Jörg Hoffmann, and Raz Nissim. Merge-and-shrink abstraction: A method for generating lower bounds in factored state spaces. *Journal of the ACM*, 61(3):16:1–16:63, 2014.

[4] Emil Keyder and Hector Geffner. Soft goals can be compiled away. *Journal of Artificial Intelligence Research*, 36:547–556, 2009.

[5] Silvia Richter and Matthias Westphal. The LAMA planner: Guiding cost-based anytime planning with landmarks. *Journal of Artificial Intelligence Research*, 39:127–177, 2010.

A Proven-optimal reference costs

Instance	Cost	Proof	Instance	Cost	Proof	Instance	Cost	Proof
small-01	154	LM-cut	medium-01	250	LM-cut	large-01	240	**exact.py**
small-02	170	LM-cut	medium-02	191	LM-cut	large-02	428	**exact.py**
small-03	146	LM-cut	medium-03	528	LM-cut	large-03	434	**exact.py**
small-04	102	LM-cut	medium-04	716	M&S	large-04	577	LM-cut
small-05	102	LM-cut	medium-05	412	LM-cut	large-05	566	LM-cut
small-06	106	LM-cut	medium-06	282	LM-cut	large-06	445	**exact.py**
small-07	122	LM-cut	medium-07	411	LM-cut	large-07	422	M&S
small-08	520	LM-cut	medium-08	280	M&S	large-08	366	**exact.py**
small-09	420	LM-cut	medium-09	330	M&S	large-09	818	M&S
small-10	136	LM-cut	medium-10	476	LM-cut	large-10	402	**exact.py**
small-11	154	LM-cut	medium-11	496	LM-cut	large-11	403	**exact.py**
small-12	158	LM-cut	medium-12	278	M&S	large-12	302	**exact.py**
small-13	120	LM-cut	medium-13	166	LM-cut	large-13	212	**exact.py**
small-14	120	LM-cut	medium-14	401	**exact.py**	large-14	460	M&S
small-15	74	LM-cut	medium-15	194	LM-cut	large-15	476	**exact.py**

Table 3: Optimal cost (driving minutes plus deferral penalties, $\lambda = 20$) of every instance, and how optimality was proven: Fast Downward A* with LM-cut or with merge-and-shrink (M&S) within 300 s, or our exact solver `exact.py` where Fast Downward ran out of time. `exact.py` also reproduces every Fast Downward optimum.